

Systems Programming and Computer Architecture

Laurin Seeholzer

September 23, 2026

Contents

1	C Programming	3
1.1	History and Toolchain	3
1.2	Control flow	3
1.3	Basic types in C	4
1.4	Operators	5
1.5	Arrays	5
1.6	Preprocessing	6
1.7	Modularity	7
1.8	Assertions	8
1.9	Style	8
2	Integers	9
2.1	operations	9

1 C Programming

1.1 History and Toolchain

C is very fast, offers preprocessing and is close to metal. What you won't find in C are objects, classes, traits, features, methods, interfaces, fancy types, exception handling.

Important 1.1. Most importantly C does not offer memory management, it uses pointers that offer direct access to memory addresses.

We'll use C99, the most commonly used variant, in this course.

Code Example 1.2

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[])
4 {
5     printf("hello, world\n");
6     return 0;
7 }
```

1.2 Control flow

Code Example 1.3

```
1 if (Boolean expression) Statement_when_true
2     else Statement_when_false
3
4 switch (Integer expression) {
5     case Constant_1: Statement; break;
6     ...
7     default: Statement; break;
8 }
9
10 for (Initial; Test; Increment) Statement
11
12 while (Boolean expression) Statement
13
14 do Statement while (Boolean expression)
15
16 goto Label
17
18 printf()
19
20 return (Expression)
```

Important 1.4. Don't use goto: It's almost never a good idea, usually argued on performance.

Remark 1.5. goto can be used to break out of multiple nested loops or to cleanup nested code.

1.3 Basic types in C

Code Example 1.6

```
1 {static, const} type variable = value;
```

Remark 1.7. The prefix **const** before a variable ensures it can not be changed lateron.

Definition 1.8. Scope: The scope of a variable in C is the Block it is defined in (lexically typed).

Remark 1.9. The prefix **static** before a variable definition ensures the value persists between function calls. Note that static limits the scope of a variable to the file/compilation unit.

Definition 1.10. Integers & Floats: Size of these differ from system to system

- **char** is always 1 Byte
- **int & float** are always 4 Bytes
- **long long & double** are always 8 Bytes

Definition 1.11. Booleans: Boolean values in C are integers where zero = false, non-zero = true

Definition 1.12. Void: Void is a type without a value and can be used for raw memory pointers or functions without returnvalue.

Definition 1.13. enums: used to enumerate/sequentially assign values to variables.

Code Example 1.14

```
1 eum { ThisIsZero, ThisIsOne, ThisIsTwo } numbers;
```

1.4 Operators

Code Example 1.15

```
1  () []
2  ! ~ ++ -- + - * & (type) sizeof
3  * / %
4  + -
5  << >>
6  < <= > >=
7  == !=
8  &
9  ^
10 |
11 &&
12 ||
13 ?:
14 = += -= *= /= %= &= ^= |= <<= >>=
15 ,
```

Remark 1.16. Early termination in C means expressions such as $A \& B$ never evaluate B if A is already evaluated to false.

Definition 1.17. Ternary Operator: ternary operators allow for concise conditional assignment.

Code Example 1.18

```
1 int a = {test}? ifTrue : ifFalse;
```

Definition 1.19. Assignment Operator: In C an assignment to a variable is not just a statement its an expressison that evaluateates to the value that is assigned.

Definition 1.20. Post- & Pre-Increment/Decement: As in many languages the difference between $a++$ and $++a$ is the value of the expression (before/after incrementing)

Important 1.21. Casting of most types to most other types in possible in C. What exactly happens is very complicated and determined by the types involved.

1.5 Arrays

Definition 1.22. Arrays are finite vectors of variables which all have the same type.

Important 1.23. The C-compiler does not check array bounds, always check them manually!

Remark 1.24. Arrays do not have to be initialized when they are defined.

Code Example 1.25

```
1 int x[N] //uninitialized
2 int x[N] {} //initialized with zeros
```

Remark 1.26. Strings are initialized as an array of chars

Code Example 1.27

```
1 char s1[6] = "hello";
2 char s2[6] = {'h', 'e', 'l', 'l', 'o', 0};
```

Remark 1.28. There are many libraries, to copy, compare, concatenate... strings in C. The language itself does not support any of these operations.

1.6 Preprocessing

Definition 1.29. Preprocessing replaces tokens in a C source file before compilation occurs. Text replacements and macros are managed using directives beginning with #.

Code Example 1.30

```
1 #define FOO BAZ           // Replaces FOO with BAZ
2 #define BAR(x) (x+3)     // Replaces BAR(x) with (x+3)
3 #undef FOO                // Undefines FOO; no replacements after this
   point
4 #define QUX               // Defines QUX as a null string
```

Remark 1.31. Complex multi-statement macros wrap code in a `do { ... }` `while(0)` loop to safely absorb trailing semicolons and prevent syntax errors when used in control flow statements (such as `if-else`).

Remark 1.32. Conditional compilation directives include or exclude blocks of code based on conditions evaluated by the preprocessor:

Code Example 1.33

```
1  #if expression
2      ...
3  #elif expression
4      ...
5  #else
6      ...
7  #endif
8
9  #ifdef FOO        // Shorthand for #if defined(FOO)
10 #ifndef BAR       // Shorthand for #if !defined(BAR)
```

Remark 1.34. Macro parameters can be manipulated using special operators:

- `#c` (Stringifying): Converts a macro parameter into a string literal.
- `x ## y` (Token Pasting / Concatenation): Merges two tokens into a single token at compile-time.

Remark 1.35. The C preprocessor provides several standard predefined macros:

- `__FILE__`: Name of the current source file.
- `__LINE__`: Current line number.
- `__DATE__` / `__TIME__`: Date and time when the preprocessor runs.
- `__STDC__` / `__STDC_VERSION__`: Standard C compliance and version info.

1.7 Modularity

Definition 1.36. Declaration vs. Definition:

- A **declaration** specifies that an entity (function prototype, type, or global variable) exists somewhere without allocating memory for it.
- A **definition** provides the actual implementation or reserves memory for the variable/function.

Definition 1.37. A **module** consists of a public interface defined in a header file (`module.h`) containing declarations and macros, and an implementation in a source file (`module.c`) containing definitions and private declarations.

Remark 1.38. Declarations can be annotated with linkage modifiers:

- **extern:** The definition is located elsewhere (in another object module or compilation unit).
- **static:** Limits the visibility/scope of a function or global variable strictly to the current compilation unit.

Code Example 1.39

```
1  /* file.h */
2  #ifndef FILE_H
3  #define FILE_H
4
5  // Declarations, function prototypes, typedefs, and macros
6
7  #endif // FILE_H
```

1.8 Assertions

Definition 1.40. An **assertion** (`assert(expression)`) is a macro that evaluates a scalar condition at runtime. If the condition evaluates to false (`0`), it prints diagnostic details (file name, line number, expression) to standard error and terminates execution via `abort()`.

Remark 1.41. Assertions are tools for developers to detect program bugs and verify invariants during development. They should not be used to catch standard runtime or user errors (such as failed file opens). Assertions can be completely disabled in production builds by defining the `NDEBUG` macro (`-DNDEBUG`).

1.9 Style

Remark 1.42. Key guidelines for writing clean C code:

- Maintain clear formatting and consistent indentation to highlight block structure.
- Use explicit parentheses when in doubt to avoid operator precedence ambiguities.
- Use descriptive names for global entities and short names for local variables.
- Eliminate magic numbers using `const`, `enum`, or `#define` constants.
- Avoid function-like macros where regular inline functions suffice.

2 Integers

Lemma 2.1. signed (twos complement) and unsigned (standard bit string) integers

Definition 2.2. Twos Complement:

Lemma 2.3. ranges of signed unsigned

Lemma 2.4. casting signed, unsigned

Remark 2.5. in operation signed and unsigned, the signed is automatically casted to unsigned

2.1 operations

Remark 2.6. logic operations behave as expected and return signed if both operands signed

Definition 2.7. shift

addition multiplication using shift division